

Guía Práctica: Implementación de un Sistema RAG (Retrieval-Augmented Generation) en Python

Prof. CERO

17 de mayo de 2026

Índice

1. Introducción al Sistema RAG	2
1.1. ¿Cómo funciona RAG?	2
2. Implementación Paso a Paso	2
2.1. Paso 1: Carga de Documentos (Loaders)	2
2.2. Paso 2: Fragmentación de Texto (Splitters)	2
2.3. Paso 3: Vectorización y Base de Datos (ChromaDB + HuggingFace)	3
2.4. Paso 4: El Cerebro del Sistema (Llama 3 vía Groq)	3
2.5. Paso 5: Construcción de la Cadena (Chain) y el Prompt	3
3. El Bucle Interactivo	4

1. Introducción al Sistema RAG

En la era de la Inteligencia Artificial, los Modelos de Lenguaje Grande (LLMs) como GPT-4 o Llama 3 son herramientas excepcionales para el razonamiento y la generación de texto. Sin embargo, su conocimiento está limitado a la información con la que fueron entrenados y son propensos a sufrir *alucinaciones* (inventar respuestas) cuando se les pregunta sobre datos privados o muy recientes.

Para solucionar esto, surge la arquitectura **RAG (Retrieval-Augmented Generation)**. Un sistema RAG actúa como un puente entre tus datos privados (apuntes, libros, repositorios) y el "cerebro" del LLM.

1.1. ¿Cómo funciona RAG?

Se divide en dos fases principales:

1. **Retrieval (Recuperación):** Cuando el usuario hace una pregunta, el sistema busca matemáticamente en la base de datos de documentos locales y extrae únicamente los fragmentos de texto más relevantes.
2. **Generation (Generación):** El sistema inyecta esos fragmentos en el modelo de lenguaje junto con la pregunta original. El LLM lee estos fragmentos y redacta una respuesta coherente basada **estrictamente** en el contexto proporcionado.

2. Implementación Paso a Paso

A continuación, desarrollaremos un script de Python que implementa esta arquitectura paso a paso utilizando la librería LangChain.

2.1. Paso 1: Carga de Documentos (Loaders)

El primer paso es leer nuestros archivos locales (como apuntes en `.tex` o `.md`). Para ello, utilizamos un `DirectoryLoader`.

```
1 from langchain_community.document_loaders import DirectoryLoader,
   TextLoader
2
3 dir_path = "/ruta/a/mis/documentos"
4 docs = []
5 for pattern in ["**/*.tex", "**/*.md"]:
6     loader = DirectoryLoader(
7         dir_path, glob=pattern, loader_cls=TextLoader,
8         loader_kwargs={"encoding": "utf-8"}
9     )
10    docs.extend(loader.load())
```

Este código escaneará todas las subcarpetas buscando archivos de texto y los cargará en la memoria RAM.

2.2. Paso 2: Fragmentación de Texto (Splitters)

Los modelos de lenguaje tienen un límite en la cantidad de texto que pueden leer a la vez (ventana de contexto). Por ende, debemos dividir nuestros documentos largos en pequeños párrafos superpuestos.

```

1 from langchain_text_splitters import RecursiveCharacterTextSplitter
2
3 text_splitter = RecursiveCharacterTextSplitter(chunk_size=2000,
4         chunk_overlap=200)
5 splits = text_splitter.split_documents(docs)

```

Aquí dividimos el texto en bloques de 2000 caracteres, permitiendo que 200 caracteres se superpongan entre bloques para no cortar ideas por la mitad.

2.3. Paso 3: Vectorización y Base de Datos (ChromaDB + HuggingFace)

Las computadoras no entienden de palabras, sino de números. Usaremos un modelo local y gratuito de **Hugging Face** para convertir nuestros fragmentos de texto en vectores matemáticos (Embeddings) y guardarlos en una base de datos vectorial llamada **ChromaDB**.

```

1 from langchain_huggingface import HuggingFaceEmbeddings
2 from langchain_chroma import Chroma
3
4 embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
5 persist_dir = "chroma_db"
6
7 # Guardar los fragmentos como vectores en disco
8 vectorstore = Chroma.from_documents(
9     documents=splits,
10    embedding=embeddings,
11    persist_directory=persist_dir
12 )

```

Esto permite que la búsqueda posterior sea instantánea.

2.4. Paso 4: El Cerebro del Sistema (Llama 3 vía Groq)

Para la fase de razonamiento y generación de texto, conectaremos nuestro sistema al modelo de lenguaje **Llama 3** utilizando la API ultrarrápida de **Groq**.

```

1 from langchain_groq import ChatGroq
2
3 # Asegurate de tener configurada la variable de entorno GROQ_API_KEY
4 llm = ChatGroq(model="llama-3.1-8b-instant", temperature=0)

```

Establecemos la temperatura en 0 para que las respuestas sean deterministas y precisas, limitando la creatividad del modelo, lo cual es ideal para un asistente académico.

2.5. Paso 5: Construcción de la Cadena (Chain) y el Prompt

Finalmente, ensamblamos todo. Le diremos al sistema cómo comportarse (Prompt), le pasaremos el buscador (Retriever) y el cerebro (LLM).

```

1 from langchain_classic.chains import create_retrieval_chain
2 from langchain_classic.chains.combine_documents import
3     create_stuff_documents_chain
4 from langchain_core.prompts import ChatPromptTemplate
5
6 # 1. Recuperador: Buscará los 6 fragmentos más relevantes a la pregunta
7 retriever = vectorstore.as_retriever(search_kwargs={"k": 6})

```

```

7
8 # 2. Instrucciones para el Asistente
9 system_prompt = (
10     "Eres un asistente académico experto. Usa los siguientes fragmentos
    de contexto "
11     "para responder a la pregunta del usuario. Si no sabes la respuesta,
    di que no lo sabes.\n\n"
12     "{context}"
13 )
14 prompt = ChatPromptTemplate.from_messages([
15     ("system", system_prompt),
16     ("human", "{input}"),
17 ])
18
19 # 3. Ensamblar la cadena RAG
20 question_answer_chain = create_stuff_documents_chain(llm, prompt)
21 rag_chain = create_retrieval_chain(retriever, question_answer_chain)

```

3. El Bucle Interactivo

Para que el script actúe como un chatbot que no se cierra tras cada pregunta, implementamos un bucle `while` infinito que espera la entrada del usuario.

```

1 print("\n;El sistema RAG está listo! Escribe 'salir' para terminar.")
2 try:
3     while True:
4         pregunta = input("\nTu pregunta: ")
5         if pregunta.lower() in ['salir', 'exit', 'quit']:
6             break
7
8         # Invocar la cadena y generar la respuesta
9         response = rag_chain.invoke({"input": pregunta})
10
11         print("\n--- RESPUESTA ---")
12         print(response["answer"])
13
14 except (KeyboardInterrupt, EOFError):
15     print("\n\nSaliendo del sistema RAG. ¡Hasta luego!")

```

¡Y listo! Con estas etapas, hemos construido un asistente académico privado, de costo cero en vectorización, y respaldado por la última tecnología en modelos de lenguaje de código abierto.